

BÁO CÁO PHÂN TÍCH MÃ NGUỒN

MSMS_Project

Firmware nút cảm biến trong mạng Wi-Fi Mesh

Nền tảng: ESP32 / ESP-IDF / FreeRTOS

Ngôn ngữ: C, CMake

Kiến trúc mạng: ESP-Mesh-Lite, Wi-Fi, UDP, UART

Phiên bản tài liệu: 1.0

Tài liệu được xây dựng từ trạng thái mã nguồn hiện tại

Tháng 07 năm 2026

Mục lục

1	Giới thiệu	1
1.1	Mục đích tài liệu	1
1.2	Vai trò của firmware	1
2	Nền tảng và tổ chức dự án	1
2.1	Công nghệ sử dụng	1
2.2	Cấu trúc thư mục	2
3	Kiến trúc phần mềm	2
3.1	Mô hình phân lớp	2
3.2	Sơ đồ khối toàn hệ thống	3
3.3	DataManager là trạng thái trung tâm	3
4	Luồng khởi động	4
4.1	Điểm vào app_main	4
5	Hệ thống cảm biến	5
5.1	Giao diện driver thống nhất	5
5.2	Các cảm biến đã đăng ký	6
5.3	Quy trình thêm cảm biến	6
6	Mạng và truyền dữ liệu	7
6.1	InternetManager	7
6.2	Hai vai trò mesh	7
6.3	Nguồn dữ liệu và luồng telemetry	7
7	Định dạng telemetry	8
7.1	Gói node hiện tại	8
7.2	Xử lý telemetry tại gateway	9
8	UI và tương tác người dùng	10
9	Các dịch vụ lỗi	10

10 Task FreeRTOS và tài nguyên	10
11 Bộ nhớ flash và lưu trữ	11
12 Build, flash và theo dõi	11
12.1 Yêu cầu	11
12.2 Các lệnh cơ bản	12
12.3 Log cần theo dõi	12
13 Chiến lược kiểm thử	12
14 Đánh giá và khuyến nghị	13
14.1 Điểm mạnh	13
14.2 Rủi ro kỹ thuật	14
14.3 Thứ tự cải tiến đề xuất	14
15 Kết luận	14
A Tập nguồn trọng tâm	14

Danh sách bảng

1	Các công nghệ chính	1
2	Cấu trúc mã nguồn chính	2
3	Danh sách cảm biến trong SensorRegistry	6
4	Ý nghĩa các trường telemetry	9
5	Vai trò các dịch vụ lõi	10
6	Một số task được tạo trực tiếp trong mã nguồn	11
7	Bảng phân vùng trong partitions.csv	11
8	Danh sách kiểm thử đề xuất	12

1 Giới thiệu

1.1 Mục đích tài liệu

Tài liệu này mô tả cấu trúc, luồng thực thi và các giao diện quan trọng của MSMS_Project. Mục tiêu là giúp người phát triển:

- hiểu quá trình khởi động firmware và quan hệ giữa các thành phần;
- xác định nơi thêm cảm biến, giao diện hoặc phương thức truyền dữ liệu;
- hiểu định dạng telemetry từ node tới gateway;
- có cơ sở kiểm thử, gỡ lỗi và bảo trì hệ thống.

Phạm vi tài liệu chỉ bao gồm firmware trong thư mục MSMS_Project. Dashboard, gateway bên ngoài và firmware của các vi điều khiển khác chỉ được nhắc đến tại điểm giao tiếp với dự án này.

1.2 Vai trò của firmware

MSMS_Project điều khiển một nút cảm biến dùng ESP32. Firmware đọc cảm biến tại ba cổng, hiển thị trạng thái trên OLED, quản lý pin, thời gian và lưu trữ, sau đó đóng gói dữ liệu thành JSON để gửi qua mạng mesh. Một thiết bị có thể chuyển sang vai trò root; khi đó dữ liệu UDP nhận từ các node được đưa vào hàng đợi và chuyển tiếp qua UART tới gateway.

2 Nền tảng và tổ chức dự án

2.1 Công nghệ sử dụng

Bảng 1: Các công nghệ chính

Thành phần	Vai trò
ESP-IDF	SDK, driver ngoại vi, Wi-Fi, NVS, SPIFFS và hệ thống build.
FreeRTOS	Task, queue, mutex, timer và điều phối thời gian thực.
ESP-Mesh-Lite	Hình thành mạng nhiều tầng và chuyển tiếp IP giữa các node.
CMake	Khai báo project và các component nội bộ.
cJSON	Phân tích và tái cấu trúc gói JSON tại root/gateway.
NVS, SPIFFS, SD card	Lưu cấu hình, tài nguyên web và dữ liệu cục bộ.

2.2 Cấu trúc thư mục

Bảng 2: Cấu trúc mã nguồn chính

Đường dẫn	Chức năng
main/	Điểm vào app_main(), khởi tạo toàn hệ thống.
component/core/	Dữ liệu trung tâm, pin, thời gian, lưu trữ, hiệu năng, xử lý mesh và UART gateway.
component/sensors/	Kiểu cảm biến, registry và lớp quản lý cảm biến.
component/drivers/	Driver phần cứng: BME280, DHT, AHT10, PMS7003, MH-Z14A, DS3231, SSD1306, PCF8574 và các driver hỗ trợ.
component/ui/	Nút nhấn, màn hình và hệ thống menu.
component/network/	InternetManager, Wi-Fi, MeshManager, Web-Socket và trang cấu hình Wi-Fi.
component/utils/	Mã lỗi và các tiện ích bit.
openwiki/	Tài liệu kiến trúc và hướng dẫn phát triển.
partitions.csv	Bảng phân vùng flash cho NVS, OTA và SPIFFS.

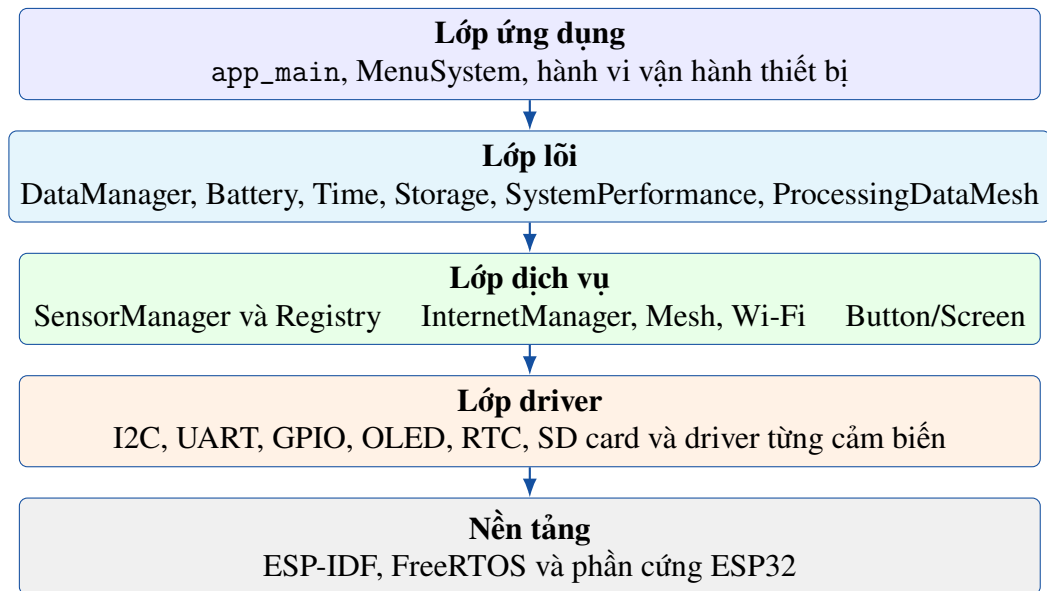
File CMake cấp dự án bổ sung sáu nhóm component vào EXTRA_COMPONENT_DIRS. Ảnh SPIFFS được tạo từ component/network/WebConfigWifi và được flash cùng firmware.

3 Kiến trúc phần mềm

3.1 Mô hình phân lớp

Hệ thống được tổ chức theo hướng component. Driver chỉ xử lý phần cứng; lớp sensor chuẩn hóa giao diện cảm biến; core điều phối dữ liệu dùng chung; network chịu trách nhiệm kết nối; UI đọc và cập nhật trạng thái thông qua DataManager_t.

Ý nghĩa các lớp. Mũi tên đi từ trên xuống thể hiện lớp phía trên sử dụng dịch vụ của lớp phía dưới. Lớp ứng dụng quyết định trình tự khởi động và hành vi thiết bị nhưng



Hình 1: Kiến trúc phân lớp của MSMS_Project

không truy cập trực tiếp thành ghi phần cứng. Lớp lõi giữ trạng thái và điều phối tác vụ. Lớp dịch vụ cung cấp các chức năng cảm biến, mạng và giao diện. Lớp driver chuyển yêu cầu chung thành thao tác I2C, UART, GPIO hoặc SPI. Cuối cùng, ESP-IDF và FreeRTOS cung cấp API hệ thống, scheduler và driver nền tảng.

3.2 Sơ đồ khối toàn hệ thống

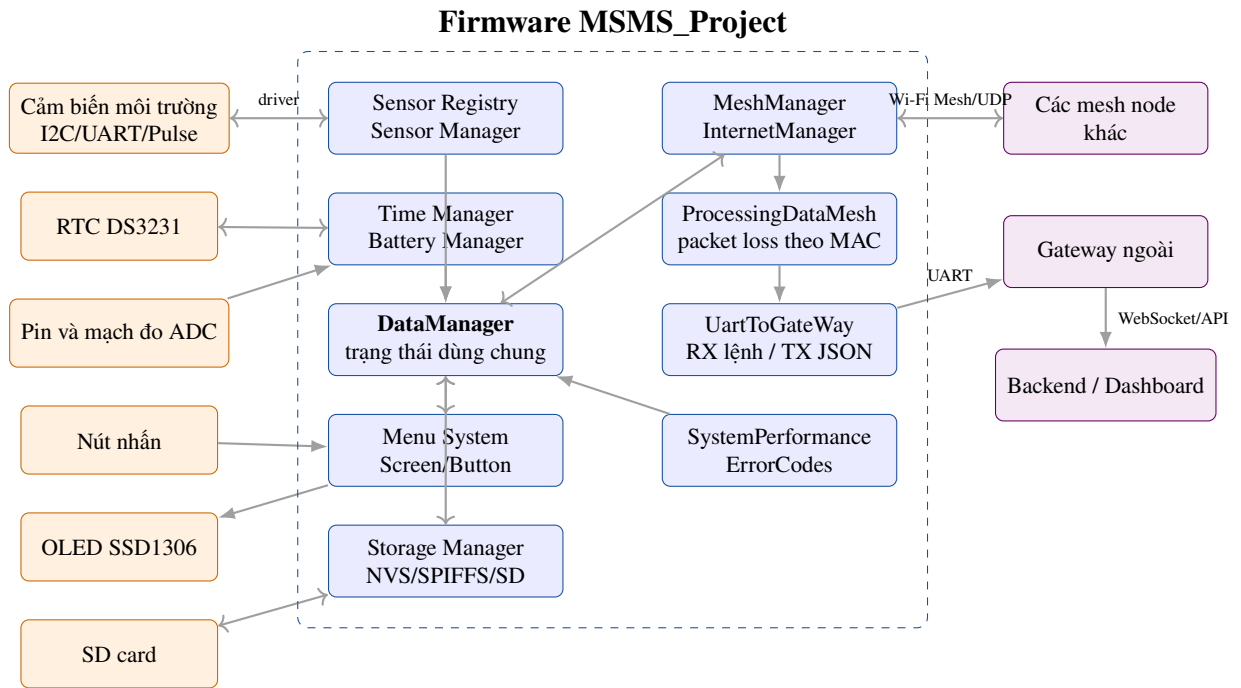
Sơ đồ khối dưới đây mô tả ranh giới giữa phần cứng, firmware trên ESP32 và các thành phần bên ngoài. Khối màu xanh nằm trong firmware MSMS_Project; các khối màu vàng là thiết bị vật lý kết nối trực tiếp; các khối màu tím thuộc tuyến gateway và hệ thống giám sát.

Trong chế độ node, đường dữ liệu chính đi từ cảm biến qua SensorManager, DataManager rồi tới MeshManager để phát UDP. Trong chế độ root, MeshManager nhận UDP từ các node khác, ProcessingDataMesh chuẩn hóa dữ liệu và task UART chuyển từng dòng JSON sang gateway. Các khối UI, lưu trữ, pin và hiệu năng hoạt động song song, cùng cập nhật hoặc đọc trạng thái trong DataManager.

3.3 DataManager là trạng thái trung tâm

DataManager_t là đối tượng dùng chung giữa các component. Cấu trúc này chứa:

- dữ liệu cảm biến và cảm biến được gán cho từng cổng;
- trạng thái nút nhấn, màn hình, Wi-Fi và mesh;
- callback, thông tin phiên bản firmware và mã lỗi;



Hình 2: Sơ đồ khối phần cứng và phần mềm của hệ thống

- handle của các task do hệ thống quản lý;
- ngữ cảnh mesh gồm queue, mutex, role, socket UDP và địa chỉ đích.

Thiết kế này giúp các module trao đổi trạng thái thuận tiện, nhưng cũng tạo ra rủi ro truy cập đồng thời. Khi bổ sung trường được nhiều task ghi, cần dùng mutex, queue hoặc vùng tới hạn thay vì đọc/ghi trực tiếp không đồng bộ.

4 Luồng khởi động

4.1 Điểm vào app_main

Trình tự khởi động trong main/main.c như sau:

1. khởi tạo NVS; nếu phân vùng hết trang hoặc sai phiên bản thì xóa và khởi tạo lại;
2. mount SPIFFS tại /spiffs;
3. khởi tạo nút nhấn và bus I2C dùng chung;
4. tạo driver OLED SSD1306 và khởi tạo ScreenManager;
5. đặt toàn bộ ba cổng về SENSOR_NONE, sau đó khởi tạo menu;
6. khởi tạo UART nối gateway;
7. khởi tạo BatteryManager và task đo pin;
8. khởi tạo SystemPerformance;
9. khởi tạo InternetManager ở chế độ mesh, vai trò ban đầu là node;

10. trên ESP32, khởi tạo RTC DS3231 và StorageManager;

11. tạo hai task điều hướng và render menu.

```

1 void app_main(void)
2 {
3     nvs_flash_init();
4     init_spiffs();
5     ButtonManagerInit(&DataManager);
6     i2cInitDevCommon();
7
8     ScreenManagerInit(&DataManager, oled_dev);
9     MenuSystemInit(&DataManager);
10    UartToGateWay_Init(&DataManager);
11
12    BatteryManager_Init(&DataManager, ADC_UNIT_1);
13    BatteryManager_StartTask(&DataManager);
14    SystemPerformance_Init(&DataManager);
15
16    InternetManager_Init(&DataManager, INTERNET_MODE_MESH);
17 }
```

Listing 1: Phần cốt lõi của quá trình khởi động

Mặc định firmware không khởi động ở chế độ Wi-Fi độc lập. Muốn thay đổi cần gọi `InternetManager_SwitchMode()` hoặc sửa tham số trong `app_main()`.

5 Hệ thống cảm biến

5.1 Giao diện driver thông nhất

Mỗi cảm biến được mô tả bởi `sensor_driver_t`, gồm tên, mô tả, đơn vị đo, số lượng giá trị, loại giao tiếp và ba callback `init/read/deinit`. Nhờ đó `SensorManager` và `MeshManager` không cần biết chi tiết thanh ghi hay giao thức của từng thiết bị.

```

1 typedef struct {
2     const char *name;
3     char description[20];
4     char unit[20];
5     bool is_init;
6     uint8_t unit_count;
7     SensorInterface interface;
```

```

8     system_err_t (*init)(SensorPort_t *port);
9     system_err_t (*read)(SensorPort_t *port, SensorData_t *data);
10    system_err_t (*deinit)(SensorPort_t *port);
11 } sensor_driver_t;

```

Listing 2: Giao diện chung của driver cảm biến

5.2 Các cảm biến đã đăng ký

Bảng 3: Danh sách cảm biến trong SensorRegistry

ID	Cảm biến	Giao tiếp	Dữ liệu chính
0	BME280	I2C	Nhiệt độ, áp suất, độ ẩm
1	MH-Z14A	UART	Nồng độ CO ₂ , nhiệt độ
2	PMS7003	UART	PM1.0, PM2.5, PM10
3	DHT22	Pulse	Nhiệt độ, độ ẩm
4	AHT10	I2C	Nhiệt độ, độ ẩm
5	DHT11	Pulse	Nhiệt độ, độ ẩm
6	HTU21D	I2C	Nhiệt độ, độ ẩm

5.3 Quy trình thêm cảm biến

1. tạo hoặc hoàn thiện driver trong component/drivers/;
2. thêm giá trị vào SensorType mà không thay đổi ID đã phát hành;
3. khai báo một sensor_driver_t với đủ callback;
4. thêm driver vào bảng trong SensorRegistry.c;
5. cập nhật CMake và menu lựa chọn cảm biến nếu cần;
6. kiểm thử khởi tạo, đọc, mất kết nối và khởi tạo lại;
7. cập nhật phía gateway để hiểu ID và thứ tự các giá trị mới.

ID cảm biến là một phần của giao thức truyền dữ liệu. Việc chèn một enum vào giữa danh sách có thể khiến dashboard diễn giải sai dữ liệu đã triển khai.

6 Mạng và truyền dữ liệu

6.1 InternetManager

InternetManager cung cấp ba chế độ: `INTERNET_MODE_NONE`, `INTERNET_MODE_WIFI` và `INTERNET_MODE_MESH`. Khi đổi chế độ, module dừng các task mạng cũ, đóng hoặc tái tạo netif, reset trạng thái mesh rồi mới khởi động stack mới. Cờ `transitioning` ngăn hai thao tác chuyển chế độ chạy đồng thời.

6.2 Hai vai trò mesh

Node: tham gia Mesh-Lite, đọc dữ liệu cảm biến, tạo JSON và gửi UDP tới IP/port của root.

Root: bind socket UDP, nhận frame từ node và đẩy vào `gateway_rx_queue`. Task UART lấy queue này và chuyển dữ liệu tới gateway ngoài.

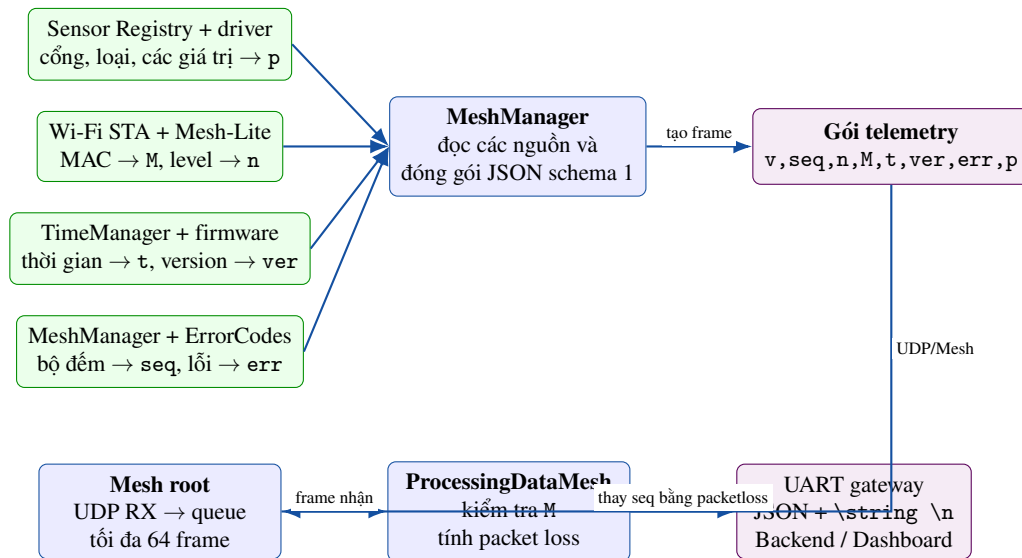
Queue nhận tại root có độ sâu 64 frame; kích thước frame được giới hạn bởi `MESH_ROOT_UDP_FRAME_SIZE`. Mutex bảo vệ việc công bố và vô hiệu hóa socket khi đổi role hoặc dọn stack mạng.

6.3 Nguồn dữ liệu và luồng telemetry

Telemetry không chỉ chứa giá trị cảm biến. Gói hoàn chỉnh được tổng hợp từ nhiều nguồn trong firmware: driver cảm biến cung cấp mảng `p`; bộ đếm MeshManager cung cấp `seq`; Mesh-Lite cung cấp `level n`; MAC Wi-Fi STA tạo trường `M`; TimeManager cung cấp `t`; phiên bản firmware tạo `ver`; ErrorCodes cung cấp mảng `err`. Sơ đồ sau chỉ rõ nguồn sinh từng nhóm trường và đường đi của gói dữ liệu.

Luồng xử lý được chia thành năm bước:

1. SensorManager gọi callback `read()` của cảm biến đã gán cho từng cổng. Chỉ kết quả đọc thành công mới được thêm vào mảng `p`.
2. MeshManager lấy MAC Wi-Fi STA làm định danh ổn định của node, lấy `level` hiện tại từ Mesh-Lite, `timestamp` từ TimeManager, phiên bản và danh sách lỗi.
3. Các giá trị được ghép thành một JSON schema 1. Node gửi frame này bằng UDP tới địa chỉ root sau một khoảng `jitter` nhằm giảm xung đột khi nhiều node phát gần đồng thời.
4. Root nhận UDP, giữ nguyên payload và đưa vào `gateway_rx_queue`. Nếu queue đầy, frame bị loại và bộ đếm `dropped_frames` tăng.



Hình 3: Nguồn hình thành trường dữ liệu và tuyến truyền telemetry

5. ProcessingDataMesh dùng cặp M/seq để tính tỷ lệ mất gói của từng node trong cửa sổ 10 giây. Sau đó UART gửi một JSON trên mỗi dòng tới gateway, từ đó dữ liệu được chuyển tới backend và dashboard.

Các nguồn dữ liệu phụ. BatteryManager, SystemPerformance, StorageManager và trạng thái mạng được lưu trong DataManager để phục vụ màn hình hoặc chức năng chẩn đoán. Chúng không tự động xuất hiện trong mảng p; muốn truyền thêm các chỉ số này phải bổ sung trường vào schema, tăng phiên bản v nếu thay đổi không tương thích và cập nhật parser phía gateway.

Lệnh UART Start Root yêu cầu thiết bị chuyển sang root nếu mesh đã sẵn sàng. Gateway gửi heartbeat Connected; nếu quá một giây không nhận heartbeat, firmware chuyển về node. Khi root không có frame trong queue, firmware gửi No node định kỳ.

7 Định dạng telemetry

7.1 Gói node hiện tại

MeshManager tạo JSON theo thứ tự khóa cố định: v, seq, n, M, t, ver, err, p.

Listing 3: Ví dụ gói telemetry schema 1

```
{
  "v": 1,
  "seq": 281,
  "n": 2,
```

```

"M": "B4:BF:E9:34:0C:CC",
"t": "2012-01-01T00:06:23",
"ver": "0.0.1",
"err": ["0x51"],
"p": [[1, 6, 27.142, 43.030]]
}

```

Bảng 4: Ý nghĩa các trường telemetry

Khóa	Kiểu	Ý nghĩa
v	số nguyên	Phiên bản schema; phải tăng khi thay đổi không tương thích.
seq	số nguyên	Bộ đếm gói, dùng phát hiện mất gói.
n	số nguyên	Level của node trong mạng mesh.
M	chuỗi	MAC Wi-Fi STA, định danh nguồn phát.
t	chuỗi	Thời gian ISO dạng YYYY-MM-DDTHH:MM:SS.
ver	chuỗi	Phiên bản firmware major.minor.patch.
err	mảng chuỗi	Các mã lỗi runtime dạng hexadecimal.
p	mảng mảng	Mỗi hàng gồm cổng, loại cảm biến và các giá trị.

Một phần tử p có dạng [port, sensor_type, value_0, value_1, ...]. Chỉ cổng đã gán cảm biến và đọc thành công mới được đưa vào gói.

7.2 Xử lý telemetry tại gateway

Schema đã được thống nhất dùng khóa M. Hàm `ProcessingDataMesh_ProcessFrame()` yêu cầu trường MAC này, dùng MAC làm khóa riêng cho bảng trạng thái packet loss và giữ nguyên M trong frame đầu ra. Gói không có M được xem là không hợp lệ thay vì gộp nhầm vào một định danh mặc định. Bộ đệm định danh node có 18 byte, đủ chứa 17 ký tự MAC và ký tự kết thúc chuỗi.

8 UI và tương tác người dùng

UI gồm ButtonManager, ScreenManager và MenuSystem. Hai task độc lập thực hiện đọc thao tác điều hướng và render màn hình. Trạng thái lựa chọn cảm biến của ba cổng được lưu trong DataManager, vì vậy menu cấu hình và luồng thu thập dữ liệu cùng sử dụng một nguồn trạng thái.

Khi mở rộng UI cần tránh thực hiện I/O chậm trong callback render. Thao tác mạng, SD card hoặc cảm biến nên được chuyển cho task dịch vụ; UI chỉ đọc bản chụp trạng thái và phát lệnh.

9 Các dịch vụ lỗi

Bảng 5: Vai trò các dịch vụ lỗi

Module	Trách nhiệm
BatteryManager	Đo điện áp, ước lượng mức pin và cập nhật nguồn cấp.
TimeManager	Làm việc với RTC DS3231 và tạo timestamp cho telemetry.
StorageManager	Ghi dữ liệu định kỳ; trên ESP32 được khởi tạo từ app_main().
SystemPerformance	Thu thập heap, tải CPU và các chỉ số runtime.
ProcessingDataMesh	Parse JSON tại gateway, tính tỷ lệ mất gói theo cửa sổ 10 giây và tạo frame đầu ra.
UartToGateWay	Nhận lệnh từ gateway, quản lý heartbeat và chuyển frame của root ra UART.
ErrorCodes	Chuẩn hóa mã lỗi và lưu lỗi vào mảng của DataManager.
PinManager	Tập trung ánh xạ chân và cấu hình ngoại vi.

10 Task FreeRTOS và tài nguyên

Các con số stack là số đơn vị mà API ESP-IDF nhận cho task trong bản mã hiện tại. Cần kiểm tra uxTaskGetStackHighWaterMark() trên thiết bị thật trước khi giảm stack.

Bảng 6: Một số task được tạo trực tiếp trong mã nguồn

Task	Stack	Priority	Chức năng
uart_gateway_rx	4096	6	Nhận heartbeat và lệnh UART.
uart_gateway_tx	4096	7	Chuyển queue mesh sang UART.
mesh_link	3072	5	Theo dõi kết nối và socket mesh.
mesh_data	4096	5	Gửi hoặc nhận telemetry UDP.
Menu navigation	4096	5	Xử lý điều hướng menu.
Menu render	4096	5	Cập nhật OLED.
Battery task	4096	5	Đọc và cập nhật thông tin pin.
Storage task	4096	4	Ghi dữ liệu định kỳ.
System performance	2048	5	Thu thập chỉ số hệ thống.

Task UART TX có priority cao hơn để tránh queue root bị đầy khi lưu lượng mesh tăng.

11 Bộ nhớ flash và lưu trữ

Bảng 7: Bảng phân vùng trong partitions.csv

Tên	Loại	Offset	Kích thước
nvs	data/nvs	0x9000	0x5000
otadata	data/ota	0xe000	0x2000
ota_0	app/ota_0	0x10000	0x1C0000
ota_1	app/ota_1	tự động	0x1C0000
spiffs	data/spiffs	tự động	0x20000

Hai phân vùng OTA cho phép cập nhật firmware theo mô hình A/B. SPIFFS có dung lượng 128 KiB và chứa tài nguyên cấu hình web. Khi mount SPIFFS thất bại, `init_spiffs()` hiện không tự format, giúp tránh mất dữ liệu ngoài ý muốn nhưng yêu cầu người vận hành xử lý lỗi phân vùng.

12 Build, flash và theo dõi

12.1 Yêu cầu

- ESP-IDF tương thích với các dependency của dự án;

- Python và toolchain Xtensa/RISC-V do ESP-IDF cung cấp;
- board ESP32 phù hợp với cấu hình target;
- cáp USB/UART và đúng cổng COM.

12.2 Các lệnh cơ bản

Listing 4: Build và nạp firmware

```
cd MSMS_Project
idf.py set-target esp32
idf.py menuconfig
idf.py build
idf.py -p COMx flash monitor
```

Chỉ chạy `set-target` khi cần đổi chip vì thao tác này có thể tạo lại cấu hình build. Các giá trị cần kiểm tra trong `menuconfig` gồm kênh mesh, IP root, UDP port, baud UART gateway, Wi-Fi mesh và bảng phân vùng.

12.3 Log cần theo dõi

- lỗi mount SPIFFS hoặc khởi tạo NVS;
- trạng thái layer mesh, parent RSSI và free heap;
- số frame UDP nhận, frame vào queue và frame bị drop;
- thống kê UART gồm frame, byte và mức sử dụng queue;
- lỗi đọc cảm biến và mảng `err` trong telemetry;
- reset reason, stack watermark và heap nhỏ nhất.

13 Chiến lược kiểm thử

Bảng 8: Danh sách kiểm thử đề xuất

Nhóm	Kịch bản	Kết quả mong đợi
Khởi động	Flash sạch, reset nóng, NVS lỗi phiên bản	Firmware khởi động, lỗi được ghi nhận, không treo.

Nhóm	Kịch bản	Kết quả mong đợi
Cảm biến	Gắn từng loại, rút nóng, dữ liệu ngoài dải	Không crash; cổng lỗi không xuất dữ liệu sai.
Mesh node	Mất parent rồi kết nối lại	Socket được phục hồi, sequence tiếp tục hợp lệ.
Mesh root	Nhiều node gửi đồng thời	Queue không tràn trong tải thiết kế; thống kê drop chính xác.
UART	Mất heartbeat Connected	Thiết bị chuyển từ root về node.
Giao thức	Gói có M, thiếu M, sequence nhảy	Định danh đúng; gói thiếu MAC bị từ chối; packet loss đúng.
Bộ nhớ	Chạy liên tục 24 giờ	Không giảm heap bất thường, không watch-dog.
OTA	Nâng cấp và rollback	Boot đúng partition, bảo toàn cấu hình cần thiết.

14 Đánh giá và khuyến nghị

14.1 Điểm mạnh

- chia component rõ ràng và có registry để mở rộng cảm biến;
- tách vai trò node/root, có queue và thống kê drop;
- có lớp InternetManager để chuyển mode và dọn tài nguyên;
- có hai phân vùng OTA, quản lý lỗi, hiệu năng và pin;
- telemetry gọn, phù hợp thiết bị nhúng và băng thông thấp.

14.2 Rủi ro kỹ thuật

1. DataManager là trạng thái toàn cục lớn; cần quy tắc rõ ràng về quyền ghi và đồng bộ giữa task.
2. JSON được ghép bằng snprintf; phải kiểm thử sát giới hạn buffer khi tăng số trường hoặc số cảm biến.
3. Một số comment nguồn có dấu hiệu sai mã hóa ký tự, gây khó bảo trì.
4. Timestamp có thể bắt đầu ở năm 2012 nếu RTC chưa được đồng bộ; phía nhận cần đánh dấu dữ liệu thời gian chưa hợp lệ.

14.3 Thứ tự cải tiến đề xuất

1. thêm test tự động để khóa chặt schema telemetry dùng M;
2. định nghĩa rõ ownership và mutex của từng nhóm trường DataManager;
3. bổ sung versioning và kiểm tra kích thước cho mọi frame;
4. thêm test tải queue root với nhiều node và giám sát stack watermark;
5. chuẩn hóa encoding toàn bộ file nguồn về UTF-8;
6. đồng bộ tài liệu trong openwiki/ với chế độ mesh mặc định.

15 Kết luận

MSMS_Project là firmware ESP-IDF theo kiến trúc component, hỗ trợ nhiều cảm biến và hai vai trò trong mạng Mesh-Lite. Luồng chính đã tách tương đối rõ giữa thu thập dữ liệu, giao diện, kết nối và chuyển tiếp gateway. Schema định danh node đã được thống nhất dùng khóa M. Điểm cần ưu tiên trước khi mở rộng hệ thống là bảo đảm truy cập DataManager an toàn giữa các task và bổ sung kiểm thử hồi quy cho giao thức. Kiến trúc hiện tại có nền tảng phù hợp để bổ sung cảm biến, OTA và các chức năng giám sát quy mô lớn hơn.

A Tập nguồn trọng tâm

Tập nguồn và nội dung nên đọc

main/main.c

Trình tự khởi tạo hệ thống.

Tệp nguồn và nội dung nên đọc

`component/core/DataManager/DataManager.h`

Trạng thái trung tâm và ngữ cảnh mesh.

`component/sensors/SensorRegistry/SensorRegistry.c`

Danh sách driver cảm biến.

`component/network/InternetManager/InternetManager.c`

Chuyển đổi Wi-Fi/Mesh và dọn tài nguyên.

`component/network/MeshManager/MeshManager.c`

Mesh, UDP, telemetry, queue và vai trò node/root.

`component/core/ProcessingDataMesh/ProcessingDataMesh.c`

Parser telemetry và tính tỷ lệ packet loss theo MAC.

`component/core/UartToGateWay/UartToGateWay.c`

Giao thức UART với gateway.

`partitions.csv`

Bố trí flash và OTA.
